

1. Introducción

Last edited time	October 14, 2026 8:55 PM
Practica	☐
Teoria	☑

Contenidos mínimos

- Importar módulos
- Formato estándar para definir funciones
- Archivos
- Listas
  - Listas por comprensión
  - Slicing de listas
- Strings
  - Concatenación implícita de strings
  - Secuencias de escape
  - Métodos
  - Variables en strings

Atributos y encapsulamiento OOP

- Encapsulamiento
  - Encapsulamiento en Python

I/O

- Archivos de texto ($\text{bytes} \leftrightarrow \text{texto}$)
- Manejo de archivos como bytes
- Context manager
- Emular archivos (`StringIO` , `BytesIO`)

Anotaciones

- Módulo `typing`
- Anotaciones dentro de clases

Contenidos mínimos

Importar módulos

- `import <modulo>`
  - o `import <modulo> as <alias>`
  - accedes a las funciones ocupando el nombre del módulo (o alias): `modulo.funcion_1(parametros)`
- `from <modulo> import <funcion>`
  - o `from <modulo> import <funcion> as <alias>`
  - accedes a las funciones directamente ocupando el nombre de las funciones: `funcion_1(parametros)`

Formato estándar para definir funciones

- Indica el tipo de dato que recibe y entrega la función
- Explica su función, los datos involucrados y cada paso de la función.

```python
def sumar(numero1: int, numero2: int) -> int:
    """
    Retorna la suma de dos números enteros.

    Args:
        numero1 (int): Primer número a sumar.
        numero2 (int): Segundo número a sumar.

    Returns:
        int: Resultado de sumar numero1 y numero2.
    """
    # Calculamos la suma de los dos números recibidos
    suma = numero1 + numero2

    # Retornamos el resultado de la suma
    return suma
```

▼ Para que una función tenga un **valor predeterminado** que se utilice en caso de no recibirlo, se debe asignar en la definición

```python
def sumar(n1: int = 0, n2: int = 0) -> int:
    """Retorna la suma de dos números."""
    print("Valor n1: ", n1)
    print("Valor n2: ", n2)
    return n1 + n2

resultado = sumar()
print(resultado)
```

Archivos

- `file.readline()` retorna la siguiente línea del archivo
- `file.readlines()` retorna en una lista todo el contenido del archivo
- `file.write(s)` escribe el string `s` en el archivo, pero no agrega un `\n` al final de la línea
- `print(s, file=<file>)` escribe el string `s` en el archivo y sí agrega el `\n` al final.

▼ `file.writelines(lista)` escribe una lista (u otro iterable) de strings al archivo de una vez. Aunque, no agrega de forma automática el `\n`

```python
with open("archivo.txt", "at") as archivo:
    archivo.writelines(f"{linea}\n" for linea in lista)
```

Para poner las `\n` tendría que añadir uno a cada elemento de la lista.

Listas

- Normalmente se consideran **homogéneas** por su similitud con los arreglos.
- `lista.append()` añade **un** elemento al final de la lista, mientras que `lista.extend()` agrega **los** elementos de un iterable (como una lista, cadena o tupla) al final de la lista actual.
- `lista.insert(<posicion>,<elemento>)`

Listas por comprensión

- **Comprensión:** *conjunto de caracteres que forman un concepto.*
- **Lista por comprensión:** *lista formada por un conjunto de objetos que cumplen con un concepto o condición en particular.*

▼ Formato

```
nueva_lista = [expresion for elemento in lista]
#ej. largo_de_bandas = [len(nombre) for nombre in bandas]
```

`elemento` es el nombre arbitrario que elegimos para referirnos a cada elemento en `lista` . A su vez, `expresion` es lo que queremos guardar en `nueva_lista`

▼ Se puede añadir una **condición**, de tal modo que solo los elementos de la lista que la cumplan sean almacenados en la nueva lista

```
nueva_lista = [expresion for elemento in lista if condicion]
#ej. nombres_cortos = [nombre for nombre in bandas if len(nombre)<10]
```

▼ Se puede ocupar múltiples declaraciones `for` o `if`

```
>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

▼ La expresión inicial de una comprensión de listas puede ser cualquier expresión, incluso otra comprensión de listas

```
>>> matrix = [
        [1, 2, 3, 4],
        [5, 6, 7, 8],
        [9, 10, 11, 12],
    ]

>>> [[row[i] for row in matrix] for i in range(4)]
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Obtiene las 4 primeras columnas de una lista de listas

Slicing de listas

- `lista[inicio:fin]` , `lista[inicio:]` , `lista[:fin]` , `lista[inicio:fin:pasos]` , `a[-1]`
  - Si el slicing abarca más de lo que tiene la lista, retorna una lista vacía. Solo lanza error si se trata de acceder a un elemento en particular (ej. `lista[800]`)

▼ `lista[:]` crea una copia (***shallow***) del arreglo completo.

- El arreglo retornado está en una nueva dirección de memoria, pero los elementos que están en este nuevo arreglo, hacen referencia a la dirección de memoria de los elementos del arreglo inicial.

- `lista[-n:]` retorna los últimos `n` elementos del arreglo
- `lista[:-n]` retorna todos los elementos del arreglo menos los últimos `n` elementos.
- `lista[::-1]` invierte una lista
  - `lista[a:b:-1]` requiere que `a` esté más a la derecha que `b`
  - `lista[:b:-1]` empieza en el último elemento



▼ Puedes ocupar slicing de listas para **reemplazar segmentos** de elementos, con elementos de otra lista

```
>>> letras
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
>>> letras[1:4] = ["z", "w"]
```

```
>>> letras
['a', 'z', 'w', 'e', 'f', 'g', 'h']
```

▼ Si ocupas pasos distintos a 1, la sublista seleccionada y la lista asignada deben tener la **misma longitud**

```
>>> a = [0, 1, 2, 3, 4, 5]
>>> a[::2] = [99, 88]
...
ValueError: attempt to assign sequence of size 2 to extended slice of size 3
```

▼ Si la sublista está vacía, no reemplaza sino que **inserta** los elementos de la otra lista

```
>>> x = [1,2,3,4]
>>> x[:0] = [5,6]
>>> x
[5, 6, 1, 2, 3, 4]
>>> x[4:4] = [9, 9]
>>> x
[5, 6, 1, 2, 9, 9, 3, 4]
```

Strings

- Secuencia inmutable de caracteres Unicode.

Concatenación implícita de strings

- Si se ponen múltiples strings de forma adyacente, separados por espacios en blanco, Python los concatena al momento de compilar.

```
>>> ejemplo = ("Tres" "strings" "juntos")
>>> print(ejemplo)
Tres strings juntos
```

- Esto es de especial utilidad para **ahorrarnos caracteres**, porque podemos escribir en múltiples líneas sin necesidad de añadir `\n` u otro signo para esto.
  - Ojo, ocupar triples comillas sí incluye los `\n` para los saltos de líneas.

▼ Esta técnica también permite **ocupar distintas comillas** para ocupar apostrofes y comillas al interior de nuestras strings.

```
>>> phrase = (
...     "Guido's talk wasn't named "  # String with apostrophes
...     '"Feeding CPython to ChatGPT"'  # String with double quotes
... )

>>> print(phrase)
Guido's talk wasn't named "Feeding CPython to ChatGPT"
```

Secuencias de escape

- `\"` , `\'` , `\n` , `\t` , `\\` (backslash)

Métodos

- La función `dir` permite obtener la lista. En particular:
  `print(dir(str))`

- `str.index(<secuencia>,<inicio>,<fin>)` retorna el indice de la primera ocurrencia de `<secuencia>` en `str` , o retorna Error. Los otros dos parámetros son opcionales.

Variables en strings

- Para poner una llave { } dentro de una f-string se ocupa una doble llave `{{ }}` para diferenciarla de la llave simple `{ }` que se usa para poner las variables.

Plantillas

▼ Se puede crear plantillas de strings usando `str.format()`

```
correo_1 = "a@ejemplo.com"
correo_2 = "b@ejemplo.com"

saludar_usuario = "Hola {}, te damos la bienvenida!"

print(saludar_usuario.format(correo_1))
print(saludar_usuario.format(correo_2))
```

▼ Si son múltiples variables o se desea ser explicito en la variable que va en cada posición, se les puede poner nombre en la plantilla.

```
saludo_entre_usuarios = "Hola {saludado}, te saluda el usuario {saludador}"

print(saludo_entre_usuarios.format(saludado=correo_2, saludador=correo_1))
```

Especificaciones de formato

- Un **especificador de formato** (*format specifier*) es una regla dentro de las f-strings de Python que indica cómo alinear, rellenar o transformar visualmente un valor al imprimirlo. Su sintaxis básica es `{variable:formato}`. Donde `formato` puede especificarse a partir de varios modificadores que siguen un orden en particular: `[relleno y alineación][signo][ancho][separador_miles][.precisión][tipo]`

▼ **Relleno y Alineación**

- `:<` alinea a la izquierda
- `:>` alinea a la derecha
- `:^` centra el texto
- `:^10` centra el texto en un espacio de 10 caracteres, rellenando con espacios.
  - `:*^10` centra el texto en un espacio de 10 caracteres, pero rellena con un carácter especial (ej. `*`).

▼ **Signo**

- `:+` fuerza a mostrar el signo (+ o -)
- `: ` (espacio) también fuerza a mostrar el signo, pero además deja un espacio en blanco antes de los números positivos para alinearlos con los negativos.

▼ **Ancho**

- Es el “primer” número que va en la secuencia de modificadores del `formato`
- Si pone `:12` va a organizar el texto en 12 caracteres y rellenará con espacios para completarlo.
- Si pone `:012` va a hacer lo mismo, pero va a poner ceros a la izquierda para completar los 12 caracteres.

▼ **Separador miles**

- `:,` agrega la “coma” como separador de miles.
- `:_` agrega el “guion bajo” como separador de miles.

▼ **Precisión**

- Punto seguido de un entero para indicar el número de decimales (o longitud de string).

▼ **Tipo**

- Define cómo se interpreta y presenta el valor (siempre va al final)
- `%` multiplica el valor por 100 y agrega el signo de porcentaje
- `b` binario
- `x` o `X` hexadecimal
- `f` numero flotante
- `d` entero decimal

Atributos y encapsulamiento OOP

- **Variables de clases y de instancia**
  - Muchas veces, en OOP, necesitamos tener un valor que sea **compartido por todas las instancias** de una clase, pero que esta a su vez sea una **variable propia de la clase**. Para esto, podemos definir una variable de clase, simplemente declarándola dentro de su definición. Luego, para utilizarla basta llamarla como si fuera un **atributo de la clase**.

▼ Ejemplo

```
class Persona:
    run = 1

    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        self.run = Persona.run
        Persona.run += 1
```

`run` es una variable de clase. En este caso se asigna como el RUN de cada persona y después se modifica para que la siguiente persona tenga un RUN diferente.

Encapsulamiento

- El **encapsulamiento** se refiere al ocultamiento de los atributos de un objeto de manera que éstos sólo puedan ser modificados mediante los métodos que el programador defina.

  ▼ Evita que un objeto que interactúa con otro pueda **observar o modificar** elementos que sean **internos al funcionamiento** del objeto.

  - ej. una persona puede conducir un auto y, por ende, afectar el cuenta kilómetros. Sin embargo, no debería ser capaz de modificar directamente este aparato.

  - Obliga a que toda la interacción sea de la manera que el programador la definió, contribuyendo a reducir la ocurrencia de errores.
- Una **interfaz** es una fachada para proteger la implementación de una clase e interactuar con otros objetos.
  - Define el **conjunto de atributos y métodos** de un objeto que son expuestos u ofrecidos por la clase para poder interactuar con otros objetos.
  - El nivel de detalles de la interfaz se denomina **abstracción.**

Encapsulamiento en Python

- En Python **todos los atributos y métodos de un objeto son públicos.**
- Por **convención**, se puede *sugerir* que un método o atributo es de uso únicamente interno, agregando un **guion bajo** al inicio del atributo o método.

    - Sin embargo, esto no evita que todavía se pueda acceder a ellos desde afuera de la clase.
- Si se pone **doble guion bajo** al inicio del atributo o método, entonces una llamada a estos (ej. `NombreDeLaClase.__atributo`) nos va a lanzar error.
    - Esto porque Python reemplaza internamente los nombres por `_NombreDeLaClase__atributo` (véase *name mangling*).
        - Es decir, si uno los llama usando este nombre, sí van a funcionar.
    - Si uno pone doble guion bajo al inicio y al final del atributo/método, entonces sí se pueden llamar de forma normal.

I/O

Archivos de texto ($\text{bytes} \leftrightarrow \text{texto}$)

- La función `open()` permite abrir archivos, pero también permite especificar cómo **codificar los bytes** de estos y qué se debe hacer si aparecen bytes inconsistentes con el formato.
    - `open(path, mode='r', encoding=None, errors=None)`
- `encoding` es el nombre de la codificación que se usa para codificar o decodificar el archivo.
    - No se ocupa con modo en bytes.
    - El valor predeterminado es **local**, por lo que pueden ocurrir errores si uno abre el archivo en otras plataformas. Por esto, es buena práctica poner `encoding='utf-8'`
- `errors` especifica cómo se deben manejar errores de (de)codificación.
    - `='strict'` lanza `ValueError` (al igual que la opción `None`)
    - `='ignore'` ignora los errores [puede llevar a perder data]
    - `='replace'` reemplaza con un marcador (ej. `?`) los lugares con errores.

Manejo de archivos como bytes

- Para usar el modo **binario** basta con añadir **b** al lado derecho del modo en la función open. En otras palabras, los modos serían `wb` , `ab` y `rb` .
    - El archivo se comporta igual que un archivo de texto, pero sin la codificación automática de byte a texto.
- Una secuencia de **bytes** se puede representar anteponiendo una **b** al inicio de un **string**. Esto se denomina **literal de bytes.**

    ▼ **Solo es posible si** los caracteres de la string pertenecen a la tabla ASCII básica (0 al 127). En caso contrario, se usa el método `str.encode(<encoding>)`

    ```
    >>> test  = b"abcde12" #tipo bytes (secuencia de datos binarios)
    >>> test2 = "abcde12"  #tipo string (caracteres unicode)

    >>> print(test)
    b'abcde12'
    >>> print(test2)
    abcde12
    ```

    - Se puede concatenar bytes ocupando `+`
- La función `bytes(source, encoding, errors)` retorna un objeto de bytes inmutable, el cual es una secuencia de enteros entre 0 y 255, incluidos.
    - Si `source` es un string, debe llevar `encoding`
    - Para retornar a un string se ocupa el método `.decode(<encoding>)`
    - Si se le pasa un entero `n` , crea un bloque de `n` bytes llenos de ceros (`\x00`)

    - Si se le pasa una lista de números, Python entiende que cada número representa el valor de un byte [0,255].

      ▼ Si está entre 0 y 127, Python muestra el caracter ASCII* asociado.

      Por ejemplo, `b'x'` para `bytes([120])`

      * ☢️ entre 0 y 31 hay caracteres de control, por lo que Python también ocupará códigos especiales o hexadecimales en esos casos.

      ▼ Al contrario, si está entre 128 y 255, muestra el byte en formato hexadecimal

      Por ejemplo, `b'\x98'` para `bytes([152])`

- Si uno está en modo `rb` , puede ocupar el método `f.read(n)` para **leer un número entero** `n` **de bytes** desde el archivo.
    - Si se no se especifica un número, o sea se llama como `f.read()` , entonces retorna un objeto con todos los bytes del archivo.

  ▼ Cada vez que se llama `f.read()` se mueve el cursor desde donde parte a leer los bytes. Si no hay bytes que leer, retorna un objeto vacío.
    - Por ejemplo, si llamamos `f.read(1)` va a leer el primer byte, pero si volvemos a llamar `f.read(2)` va a leer el 2do y 3er bytes (porque habrá comenzado desde el 2do).
    - Para modificar la posición del cursor a su posición inicial en el archivo, puede ocupar `file.seek(0)`

Context manager

- Mientras está abierto un archivo pueden ocurrir excepciones, por esto se puede ocupar un bloque **try** (donde van las cosas que quisiéramos hacer) y un bloque **finally** (donde van las cosas que tienen que ocurrir, incluso si surge un error) para asegurar que se cierre el archivo.

▼ Un **context manager** se encarga de realizar ambos bloques **try/finally** por debajo. En concreto, para activarlo basta con abrir la sentencia con `with`

```
with open("data/IO_archivo_ejemplo_3", "r") as file:
    contenido = file.read()
```

- Los métodos `__enter__` y `__exit__` son los responsables de transformar a los archivos en un *context manager.*
    - Si uno agrega estos métodos a una clase, entonces la puedes llamar con `with`

  ▼ En el caso de los archivos, estos métodos se encargan de abrir y cerrar el archivo. Sin embargo, dependiendo de cómo los definamos en nuestras clases, podemos especificar que hagan otras acciones.

```
import string
import random
from types import TracebackType
from typing import List, Optional, Type


class StringUpper:

    def __init__(self) -> None:
        print("1. Inicizalizando context manager")
        self.data = []

    def __enter__(self) -> List[str]:
        print("2. Abriendo context manager y definiendo qué elemento será el `as XXXX`")
        return self.data

    def __exit__(
```

```
        self,
        exc_type: Optional[Type[BaseException]],
        exc_value: Optional[BaseException],
        traceback: Optional[TracebackType],
    ) -> None:
        print("5. Cerrando context manager")
        i = 0
        for char in self.data:
            self.data[i] = char.upper()
            i+= 1



with StringUpper() as s_upper:
    # s_upper será una lista según lo definido en "__enter__"
    print("3. Primera línea dentro del context manager")

    for i in range(20):
        # Aquí seleccionamos, en forma aleatoria, un ascii en minúsculas
        # y las agregamos a la lista
        s_upper.append(random.choice(string.ascii_lowercase))

    print("\t" + str(s_upper))
    print("4. Última línea del context manager")

print("\t" + str(s_upper))
```

Emular archivos (`StringIO` , `BytesIO`)

- En el módulo `io` existen las clases `StringIO` y `BytesIO` que permiten tratar datos en la memoria RAM (strings o bytes) como si fueran archivos físicos en el disco.
    - Evita la necesidad de crear archivos temporales.

▼ Ejemplo

```
from io import StringIO, BytesIO


# Aquí simulamos tener un archivo que contiene el string dado
file_in = StringIO("información como texto y más")

# Aquí simulamos un archivo de Bytes para escribir la información
file_out = BytesIO()

char = file_in.read(1)
while char:
    file_out.write(char.encode("ascii", "ignore"))
    char = file_in.read(1)

buffer_ = file_out.getvalue()
print(buffer_)
```

- `.getvalue()` se usa en objetos simulados en la RAM (como `StringIO` y `BytesIO`). Devuelve todo el contenido guardado en el búfer como una cadena de texto o de bytes. Sin importar la posición del cursor de lectura.

Anotaciones

- Agregar comentarios y anotaciones permite referenciar código antiguo y entender más rápido el código, en particular si es de un tercero. Además, ayuda a las IDEs para que sugieran código y alerten posibles errores.
- Una **anotación** permite declarar el tipo esperado de una variable, argumento o atributo. En concreto, el tipo puede ser uno de los de Python (`str` , `int` , `float` , `bool`) o una de nuestras propias clases (ej. `Persona`).

  ▼ Para crear una basta con escribir el nombre del elemento, seguido de `:` y el tipo.

  - `numero_random: int = 8`
  - `palabra: str = "hola"`

  ```
  def recibir_llamada(numero: int, nombre: str = "Desconocido"):
      print(f"Llamada entrante de {nombre}. Número {numero}")
  ```

  ▼ También se puede indicar el tipo de variable que retorna una función poniendo `->` después de la declaración de los argumentos.

  ```
  def direccion(calle: str, numero: int) -> str:
      return f'{calle.strip().title()} {numero}'
  ```

  - ⚠️ las anotaciones y alertas de IDEs no impiden que un código se ejecute.

Módulo `typing`

- Entrega tipos adicionales para ocupar en anotaciones.
- `Any` indica que el tipo de una variable no está restringida
- `Union[tipo1, tipo2, ...]` permite decir que una variable puede ser de más de un tipo distinto.

  ▼ También se puede escribir con el operador `|`

  ```
  numero: Union[int,float] = 8
  # equivale a decir
  numero: int | float = 8
  ```

▼ `Callable` permite declarar que una variable contiene la referencia a una función. Si lo desea, puede añadir una lista de los tipo de argumentos y de retorno que entrega la función, usando `Callable[[tipo_argumento,...], tipo_retorno]`

```
from typing import Callable


funcion_dividir: Callable[[Union[int, float], Union[int, float]], float] = dividir_
numeros
funcion_dividir(1.0, 2)
```

Basta con asignarle el nombre de la función para que una variable la referencie.

- `Optional` permite declarar que una variable es de algún tipo o `None`
  - Por ejemplo, `Optional[int]` equivale a `int | None`

▼ Para definir el tipo de elementos contenidos en una lista, este módulo ofrece `List[]`

```
from typing import List


lista: List[int] = [1, 2, 3, 4, 5, 6]
```

Anotaciones dentro de clases

- Python evalúa los tipos al iniciar el programa, por lo que no es posible ocupar un tipo personalizado en la definición de su propia clase.

▼ Para evitar ese error y conservar el tipo en la definición de su clase, se puede importar `annotations` del módulo `__future__` . Esta permite postergar las evaluaciones hasta que todos los tipos ya existan.

```
from __future__ import annotations  # Esto habilita la evaluación postergada de las anotaciones
from typing import Optional


class NuevaPersona:
    def __init__(self: NuevaPersona) -> None:
        self.bff: Optional[NuevaPersona] = None

    def asignar_bff(self: NuevaPersona, otro: NuevaPersona) -> None:
        self.bff = otro
```

▼ Alternativamente, en lugar de ocupar ese módulo, también puedes poner el nombre de la clase **entre comillas** y tiene el mismo efecto.

```
class PerroAmigable:
    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre
        self.edad = edad

    def saludar_amigo(self, amigo: "PerroAmigable") -> None:
        print("¡Woof!")
        print("¡Hola " + amigo.nombre + "!")
```

- Desde Python 3.14 las anotaciones se evalúan de forma diferida y esto deja de ser necesario.